QR Code Recognition Control Action (No-PTZ)

Experiment Objective

The camera (fixed bracket) scans QR codes, recognizes the encoded commands (forward/back/left/right/turnleft/turnright/stop), and controls the car to execute the corresponding chassis actions. The No-PTZ version has no pan-tilt servo; the camera is fixed facing forward and does not track or align.

Experiment Principle

1. Keyword Explanation

Keyword	Description
Visual Control	Camera captures image → recognize target content → map to chassis control commands, forming the "perception → decision → execution" chain
QR Code Command Mapping	7 normalized commands (forward/back/left/right/turnleft/turnright/stop) → predefined linear velocity + angular velocity combinations
pyzbar / OpenCV QR	Dual-backend detection: pyzbar is based on the zbar library (node default), OpenCV uses the built-in QRCodeDetector (launch default)
Joy Override	Subscribes to the `/JoyState` topic; when the joystick is active, the node automatically stops publishing `/cmd_vel` and yields control
Command Hold Timeout (hold_timeout)	After the QR code is lost, the last valid command continues to be executed within `qr_command_hold_timeout_sec`; the car stops when the timeout expires
Buzzer Feedback	Can trigger the buzzer (`/beep` topic) on unknown commands
QoS	best_effort (firmware-side sensor data uplink + cmd_vel downlink), reliable (ROS2 internal forwarding)
TF Coordinate Frames	odom → base_footprint → base_link (odometry EKF fusion chain)

2. Technical Architecture

The node uses a **dual-timer architecture** (image processing 30Hz + chassis control 20Hz), decoupling image acquisition from chassis control:

```
/camera/image_raw (BEST_EFFORT)
  → qr_control_node
    ├── Image processing timer: image conversion → flip/rotate → pyzbar/OpenCV QR detection → state update
    └── Chassis control timer: QR command mapping → /cmd_vel (BEST_EFFORT) → firmware execution
/JoyState (Bool)
  → Joystick override → zero velocity + buzzer off
/beep (UInt16)
  ← Unknown command buzzer alert
```

Key differences from the PTZ version (`qr_control_ptz_node`):

- No `/cmd_ptz_angle` publisher — no pan-tilt servo control
- No initial pan-tilt angle parameters (`initial_horizontal_deg` / `initial_pitch_deg`)
- The camera is on a fixed bracket facing forward, with no pan-tilt tracking logic

3. Core Topics Quick Reference

Firmware Uplink (Sensor Data)

Topic	Message Type	QoS	Description
`/camera/image_raw`	`sensor_msgs/Image`	best_effort	ESP32 camera raw image frames

Firmware Downlink (Control Commands)

Topic	Message Type	QoS	Field	Range	Direction
`/cmd_vel`	`geometry_msgs/Twist`	best_effort + keep_last(1)	`linear.x`	-0.30~0.30 m/s	`+=` forward, `-=` backward
			`angular.z`	-1.50~1.50 rad/s	`+=` turn left, `-=` turn right
`/beep`	`std_msgs/UInt16`	reliable	`data`	0/1	1=buzzer on

Usage example:

```
# Forward
ros2 topic pub /cmd_vel geometry_msgs/msg/Twist "{linear: {x: 0.3}, angular: {z: 0.0}}" -1
# Turn right
ros2 topic pub /cmd_vel geometry_msgs/msg/Twist "{linear: {x: 0.3}, angular: {z: -1.5}}" -1
# Stop
ros2 topic pub /cmd_vel geometry_msgs/msg/Twist "{linear: {x: 0.0}, angular: {z: 0.0}}" -1
```

ROS2 Internal Topics

Topic	Message Type	QoS	Source	Description
`/JoyState`	`std_msgs/Bool`	reliable	Joystick node	=true when the joystick is active; qr_control_node automatically stops

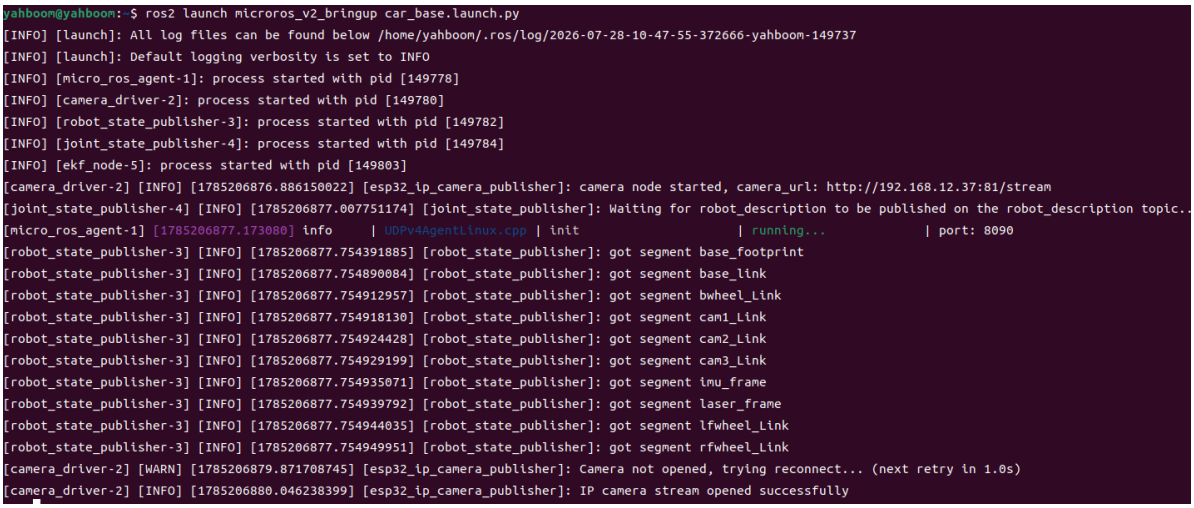
Experiment Steps

1. Hardware Preparation

Hardware	Requirement
PTZ version	❌ This section is for the No-PTZ version
No-PTZ version	☑
Required peripherals	ESP32 camera (fixed bracket)

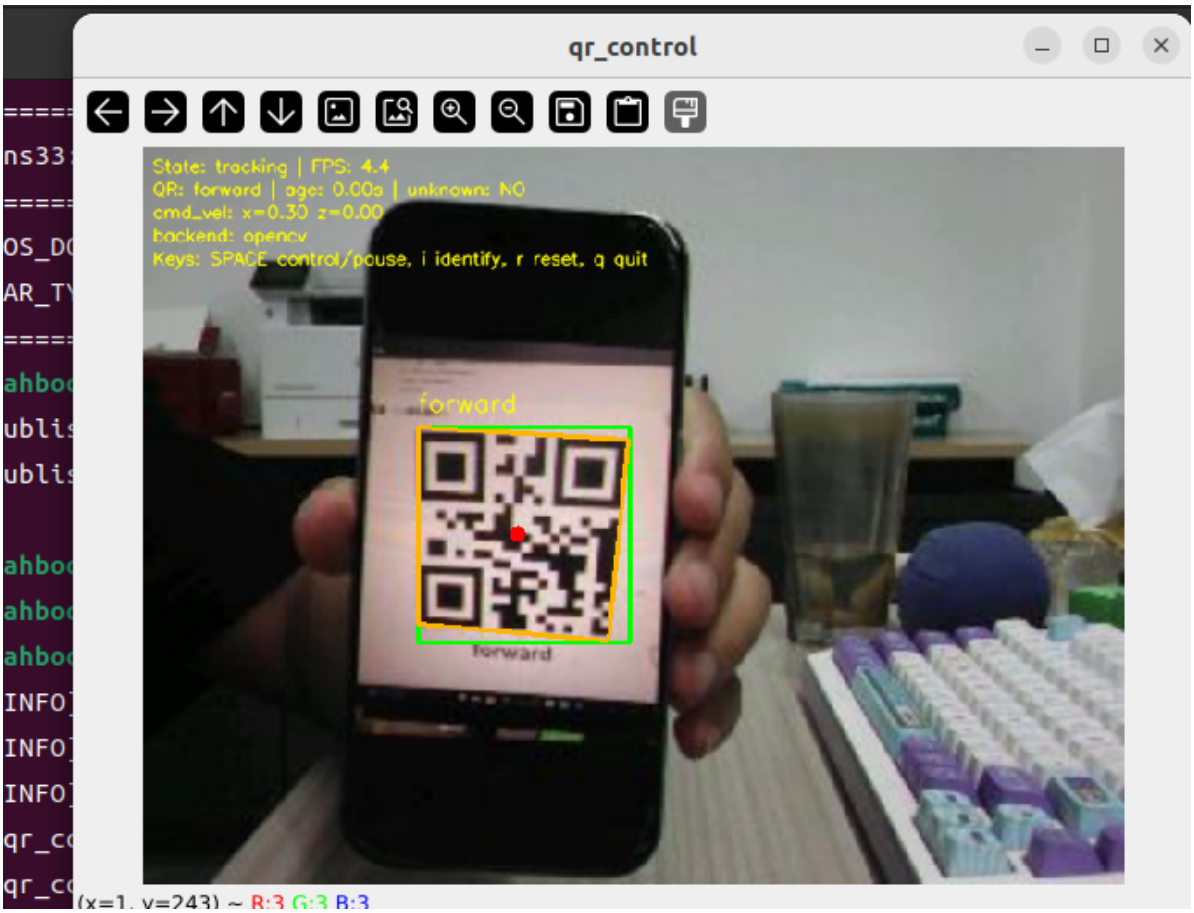
2. Start Agent + Camera + Joint Model (Terminal 1)

```
ros2 launch microros_v2_bringup car_base.launch.py
```



3. Start QR Code Control (Terminal 2)

```
ros2 launch microros_v2_ptz_visual_control_pkg qr_control_ptz.launch.py
```



5. Operation Instructions

Interaction

Key	Function	Description
Space	Control switch	Enable/pause QR code control; publishes zero velocity while paused
i / I	Recognition mode	Recognition display only, does not decode to control the chassis
r / R	Reset state	Clear the current QR code cache and stop control
q / Q / ESC	Quit	Stop + `os._exit(0)`

On-screen overlay info: status (identify/tracking/lost/paused), frame rate, QR code payload, age since last recognition (seconds), current cmd_vel value, QR detection backend. A vertical center line is drawn on the frame.

QR Code Command Mapping

Normalized command	Linear velocity (m/s)	Angular velocity (rad/s)	Behavior
`forward`	`forward_linear_speed` (0.30)	0	Drive straight forward
`back`	`back_linear_speed` (-0.30)	0	Drive straight backward
`left`	0	`rotate_angular_speed` (1.00)	Rotate left in place
`right`	0	-`rotate_angular_speed` (-1.00)	Rotate right in place
`turnleft`	`turn_linear_speed` (0.30)	`turn_angular_speed` (1.50)	Forward + turn left
`turnright`	`turn_linear_speed` (0.30)	-`turn_angular_speed` (-1.50)	Forward + turn right
`stop`	0	0	Stop

Safety and Boundary Handling

- **pyzbar unavailable**: automatically falls back to the opencv backend (launch defaults to opencv)
- **Joystick override priority**: `/JoyState=true` → immediately zero velocity + buzzer off, resumes after release
- **Unknown command handling**: when `unknown_command_stop=true`, an unknown payload triggers a stop, optionally with an `enable_beep` buzzer alert
- **Command hold timeout**: stops automatically 1.5s after the code is lost, preventing "ghost commands"
- **No pan-tilt, no tracking**: the position of the QR code in the frame does not affect command execution; no alignment is needed
- **Stop before quitting**: publishes zero velocity + buzzer off, then `os._exit(0)`

Experiment Result

- **No QR code + control enabled**: the frame shows "QR: None", `cmd_vel` is zero velocity, status is `lost`
- **QR code within hold_timeout**: continuously re-publishes the last decoded valid command
- **QR code = forward**: the car drives straight at 0.30 m/s
- **QR code = turnleft**: 0.30 m/s forward + 1.50 rad/s left turn
- **QR code lost >1.5s**: automatically stops, status is `lost`
- **Unknown QR code**: stops when `unknown_command_stop=true`, optional buzzer
- **Joystick active**: stops immediately, resumes after release
- **Multi-code scene**: selects the largest code with the "largest" strategy, draws green polygon + red center point + payload annotation

Code Explanation

> Source paths:

> - Launch: `~/microros_ws/src/microros_v2_no_ptz_visual_control_pkg/launch/qr_control.launch.py`
> - Node source: `~/microros_ws/src/microros_v2_no_ptz_visual_control_pkg/microros_v2_no_ptz_visual_control_pkg/qr_control_node.py`

1. Core Node Analysis

`qr_control_node` uses a dual-timer architecture, decoupling image processing from chassis control:

- **Image callback** (`image_callback`): only caches the latest raw image without processing, avoiding blocking
- **Image processing timer** (30Hz, `process_latest_image`): takes the latest cached frame → image conversion → orientation correction → QR detection → state update
- **Chassis control timer** (20Hz, `publish_chassis_control_command`): reads the QR state → command mapping → publishes `/cmd_vel`

Constructor — declares parameters, creates publishers/subscribers/timers:

```
# Source file:
~/microros_ws/src/microros_v2_no_ptz_visual_control_pkg/microros_v2_no_ptz_visual_control_pkg/qr_control_node.py

def __init__(self) -> None:
    """Initialize the QR control node."""
    super().__init__(DEFAULT_NODE_NAME)

    # image_lock: Image cache mutex.
    self.image_lock = threading.Lock()
    # qr_state_lock: QR state mutex.
    self.qr_state_lock = threading.Lock()

    # latest_image_msg: Latest raw Image message.
    self.latest_image_msg: Optional[Image] = None
    # latest_image_sequence_id: Latest image sequence ID.
    self.latest_image_sequence_id = 0
    # processed_image_sequence_id: Processed image sequence ID.
    self.processed_image_sequence_id = 0

    # control_state: Current control state.
    self.control_state = CONTROL_STATE_IDENTIFY
    # control_is_enabled: Whether QR control is enabled.
    self.control_is_enabled = False
    # joy_is_active: Joystick override flag.
    self.joy_is_active = False

    self.declare_and_get_parameters()
    self.control_is_enabled = self.auto_start_control
    self.control_state = CONTROL_STATE_TRACKING if self.control_is_enabled else
CONTROL_STATE_IDENTIFY

    # image_subscription: Raw image subscriber (QoS: KEEP_LAST, BEST_EFFORT)
    self.image_subscription = self.create_subscription(
        Image, self.image_topic, self.image_callback,
```

```
        QoSProfile(history=QoSHistoryPolicy.KEEP_LAST, depth=1,
                   reliability=QoSReliabilityPolicy.BEST_EFFORT,
                   durability=QoSDurabilityPolicy.VOLATILE),
    )
    # joy_state_subscription: Joystick state subscriber
    self.joy_state_subscription = self.create_subscription(
        Bool, self.joy_state_topic, self.joy_state_callback, 10,
    )
    # cmd_vel_publisher: Chassis velocity publisher (QoS aligned with the ESP32
firmware BEST_EFFORT)
    self.cmd_vel_qos = QoSProfile(
        history=QoSHistoryPolicy.KEEP_LAST, depth=1,
        reliability=QoSReliabilityPolicy.BEST_EFFORT,
        durability=QoSDurabilityPolicy.VOLATILE,
    )
    self.cmd_vel_publisher = self.create_publisher(Twist, self.cmd_vel_topic,
self.cmd_vel_qos)
    # beep_publisher: Buzzer publisher
    self.beep_publisher = self.create_publisher(UInt16, self.beep_topic, 10)

    # image_processing_timer: Image processing timer (30Hz)
    self.image_processing_timer = self.create_timer(
        1.0 / max(self.image_processing_rate_hz, MINIMUM_TIMER_RATE_HZ),
        self.process_latest_image,
    )
    # control_timer: Chassis control timer (20Hz)
    self.control_timer = self.create_timer(
        1.0 / max(self.control_rate_hz, MINIMUM_TIMER_RATE_HZ),
        self.publish_chassis_control_command,
    )

    # dynamic_parameter_callback_handle: Dynamic parameter callback (rqt tuning)
    self.dynamic_parameter_callback_handle =
self.add_on_set_parameters_callback(
        self.on_dynamic_parameter_update
    )
```

Chassis control timer callback — publishes cmd_vel at a fixed rate:

```
# Source file:
~/microros_ws/src/microros_v2_no_ptz_visual_control_pkg/microros_v2_no_ptz_visua
l_control_pkg/qr_control_node.py

def publish_chassis_control_command(self) -> None:
    """Publish chassis control command at a fixed rate."""
    if not self.is_node_running:
        return

    # Joystick override takes priority: immediate zero velocity + buzzer off
    if self.enable_joy_override and self.joy_is_active:
        self.publish_zero_twist()
        self.publish_beep(False)
        return

    # Control not enabled: zero velocity
    if not self.control_is_enabled:
        self.publish_zero_twist()
```

```
            self.publish_beep(False)
            return

        with self.qr_state_lock:
            local_qr_is_valid = self.qr_is_valid
            local_qr_payload = self.qr_payload
            local_latest_qr_update_time_sec = self.latest_qr_update_time_sec

        # QR code lost + timeout: stop
        qr_age_sec = time.perf_counter() - local_latest_qr_update_time_sec
        if (not local_qr_is_valid or qr_age_sec > self.qr_command_hold_timeout_sec)
and self.stop_when_qr_lost:
            self.control_state = CONTROL_STATE_LOST
            self.publish_zero_twist()
            self.publish_beep(False)
            return

        # Normalize the command → map to a Twist
        normalized_command = self.normalize_qr_command(local_qr_payload)
        twist_command = self.build_twist_from_qr_command(normalized_command)
        if twist_command is None:
            # Unknown command: stop + optional buzzer
            self.unknown_command_is_active = True
            if self.unknown_command_stop:
                self.publish_zero_twist()
            self.publish_beep(True)
            return

        self.unknown_command_is_active = False
        self.publish_beep(False)
        self.cmd_vel_publisher.publish(twist_command)
        self.previous_twist_command = twist_command
```

Key logic: `build_twist_from_qr_command` maps the 7 normalized commands to a `Twist` message. `normalize_qr_command` strips spaces/underscores/hyphens from the raw payload and lowercases it before matching.

2. Key Parameters

> Parameter definition files:
>
> - Launch: `~/microros_ws/src/microros_v2_no_ptz_visual_control_pkg/launch/qr_control.launch.py`
> - Node: `~/microros_ws/src/microros_v2_no_ptz_visual_control_pkg/microros_v2_no_ptz_visual_control_pkg/qr_control_node.py`

Parameter	Type	Default	Description
`forward_linear_speed`	float	`0.30` m/s	Linear velocity for the forward command
`back_linear_speed`	float	`-0.30` m/s	Linear velocity for the back command
`rotate_angular_speed`	float	`1.00` rad/s	In-place rotation angular velocity for left/right
`turn_linear_speed`	float	`0.30` m/s	Linear velocity for the turn commands
`turn_angular_speed`	float	`1.50` rad/s	Angular velocity for the turn commands
`qr_detector_backend`	str	`opencv`	Detection backend (launch defaults to opencv, node declaration defaults to pyzbar)
`qr_select_strategy`	str	`largest`	Multi-code strategy: largest / nearest_center / first
`qr_command_hold_timeout_sec`	float	`1.5` s	Command hold time after the code is lost
`control_rate_hz`	float	`20.0` Hz	Chassis control publish rate
`image_processing_rate_hz`	float	`30.0` Hz	Image processing rate
`auto_start_control`	bool	`false`	Automatically enable control at startup
`stop_when_qr_lost`	bool	`true`	Stop when the code is lost beyond the timeout
`unknown_command_stop`	bool	`true`	Stop on unknown command
`enable_joy_override`	bool	`true`	Joystick override switch
`enable_beep`	bool	`false`	Buzzer switch for unknown commands

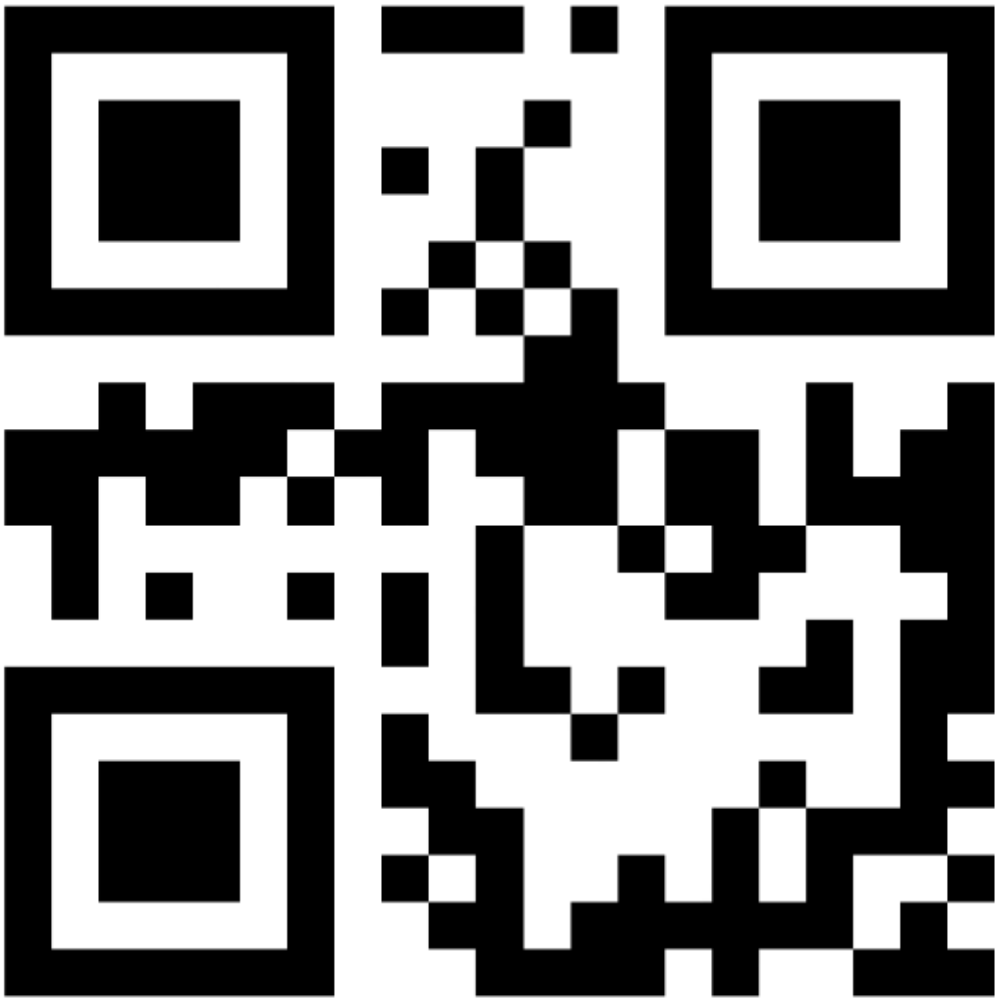
3. Node Description

Node	Package	Description
`qr_control_node`	`microros_v2_no_ptz_visual_control_pkg`	QR code recognition + control, dual-timer architecture

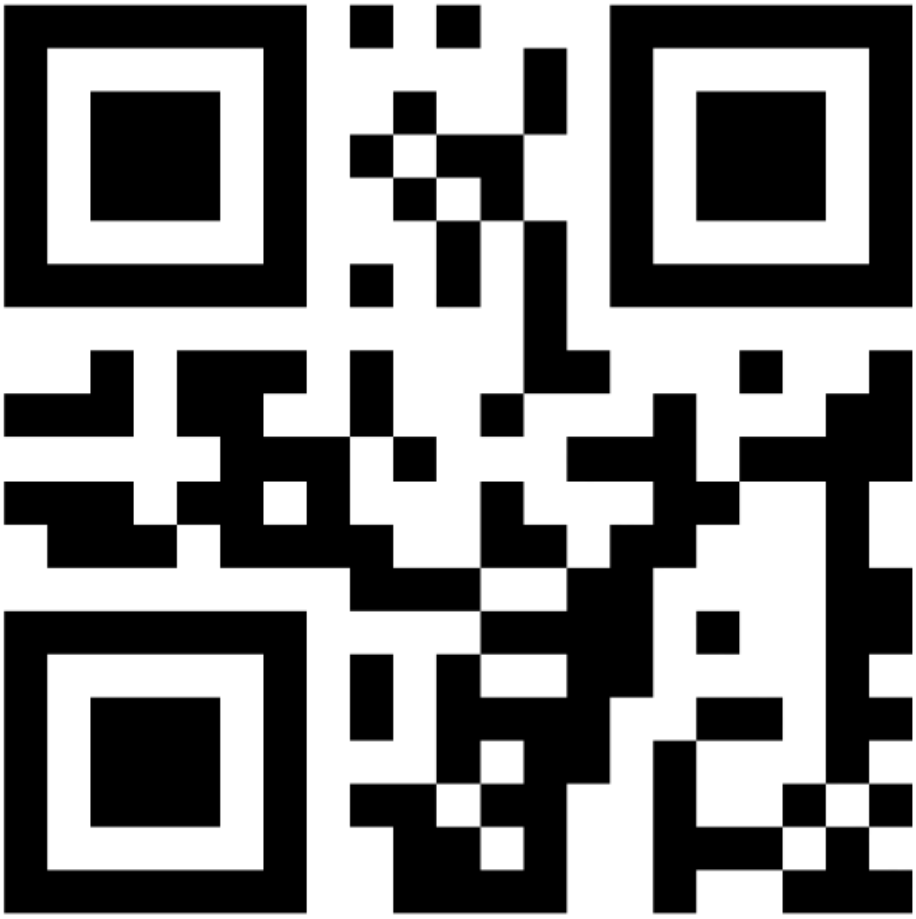
Common Problems

Problem	Cause	Solution
QR code not recognized	Insufficient lighting or blurred QR code	Ensure the QR code is clear and evenly lit
Scanned but no action	`auto_start_control=false`, needs to be enabled manually	Press Space to enable control
Car runs erratically after the code is lost	`qr_command_hold_timeout_sec` is too large	Reduce the hold timeout (e.g., 0.5s)
pyzbar unavailable	Missing libzbar0	Install `sudo apt install libzbar0`, or launch already defaults to opencv

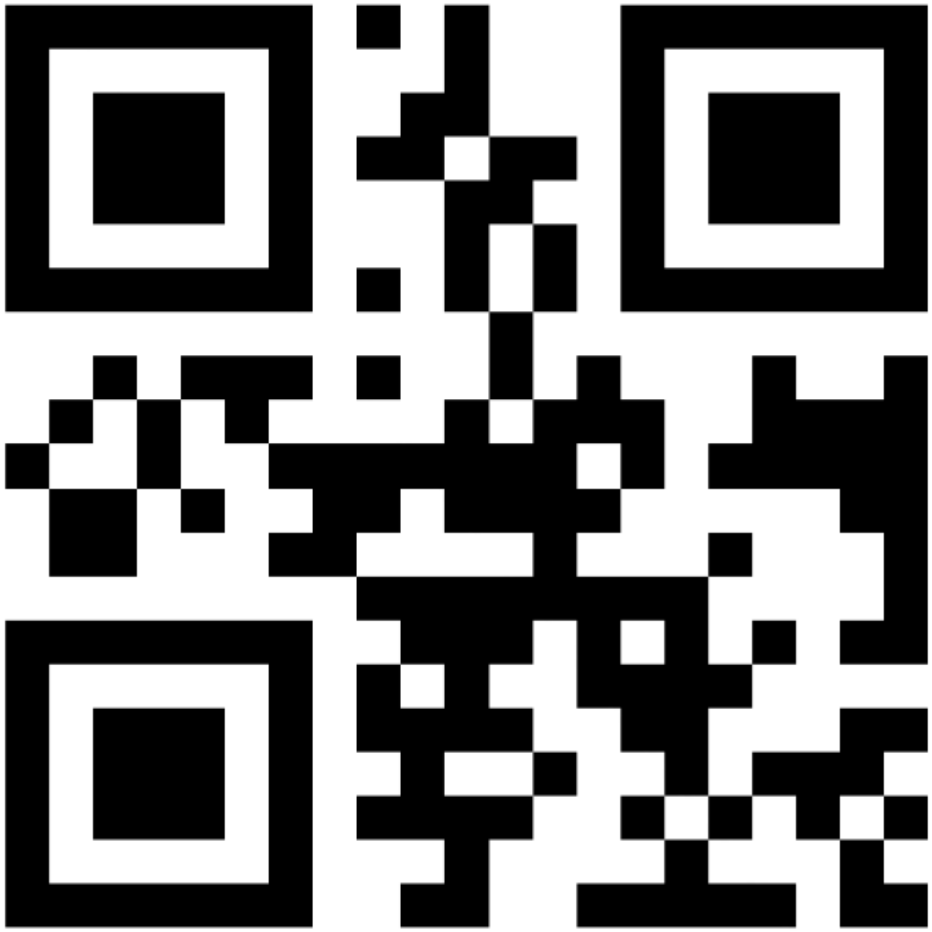
forward



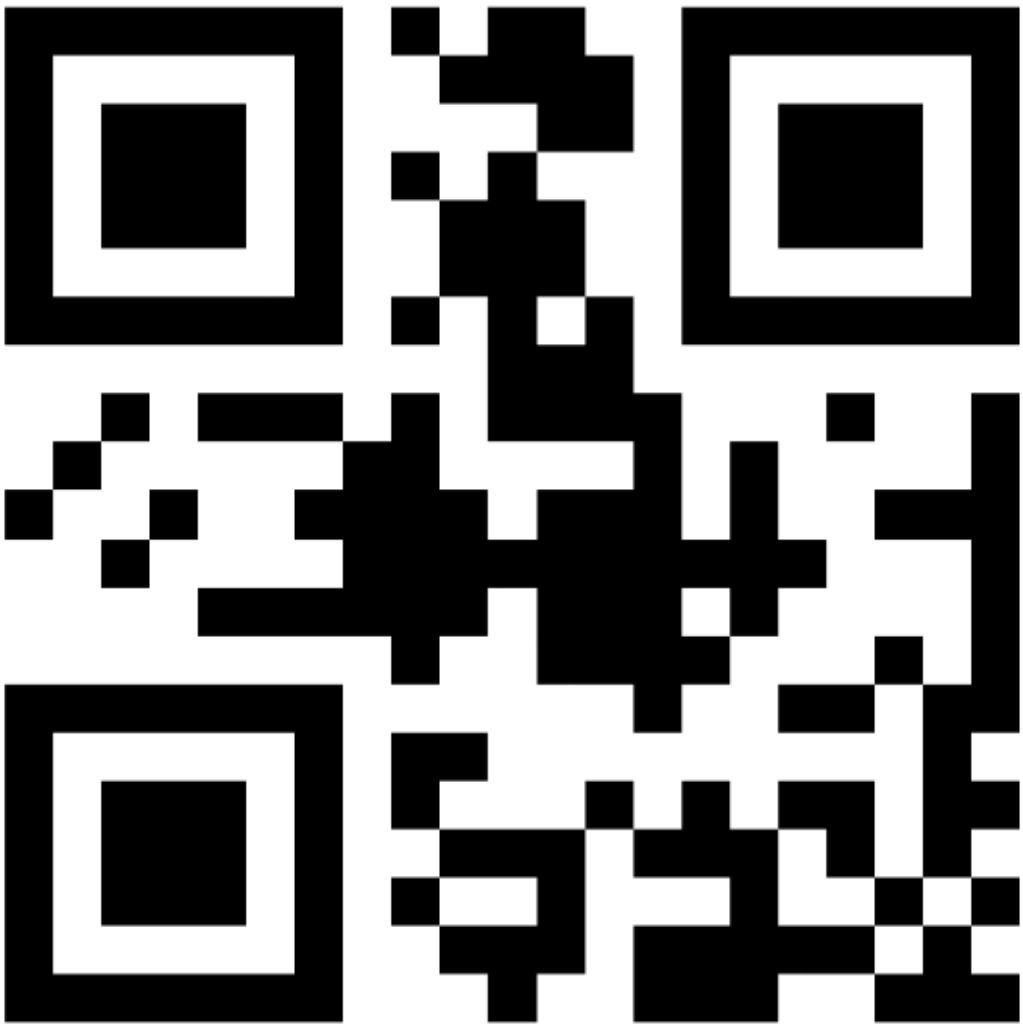
back



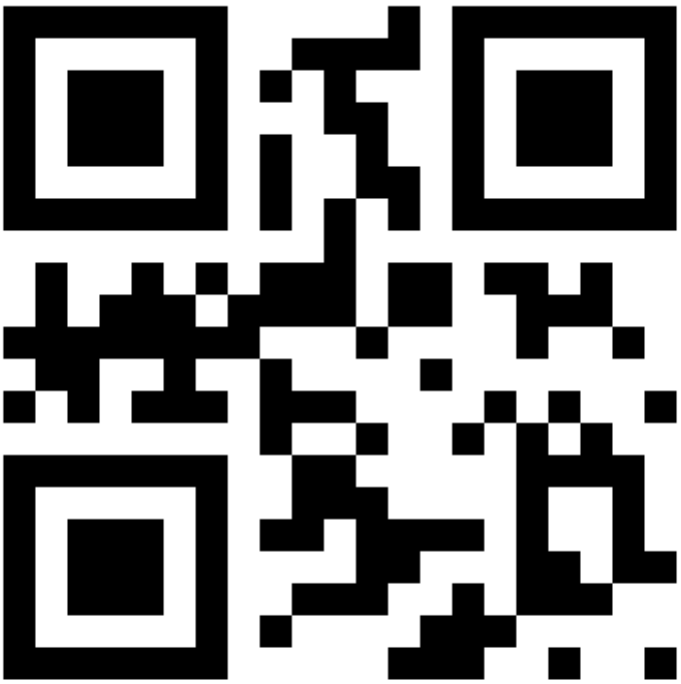
left



right



stop



turnright

turnleft